Thornwood Rescue: Strategist's Debrief

Muhammad Wasay Tariq	24I-3071
Muhammad Rayyan Hassan	24I-3000
Zain Bin Jabir	24I-3130

Assignment 02 (SE AI). Every number below comes from `thornwood.py` (the Owl Cipher) or from `debrief_runs.py`, which reruns the same functions with extra instrumentation and asserts that each rerun reproduces `thornwood.py` exactly. The extra runs are not part of the Owl Cipher. Mission status: **SUCCESS** (MM_VALUE = 1).

1. Kaito: why did I get stuck on that first hill? *(hill climbing)*

Steepest ascent from (0,0) moved to the highest neighbour only if it was *strictly* higher, checking up, right, down, left:

`(0,0)=52 → (0,1)=58 → (1,1)=64 → (2,1)=70`: 3 steps. At (0,0), right and down tie at 58, so right wins because it comes first.

Neighbourhood of (2,1)

58	**64**	58
64	**70**	**64**
58	**64**	58

The exact spot is (2,1) with value 70. All four neighbours are 64 (Δ = −6 each way): up (1,1), right (2,2), down (3,1) and left (2,0). No neighbour is higher, so the climb stops. That makes (2,1) a **strict local maximum**. The whole map has only two of them: (2,1)=70 and the real summit (6,6)=95.

The map is two hills separated by a low band through columns 3–4 of the upper rows (values 40–53). A flood fill over the map shows that every route from (2,1) to (6,6) must pass through a cell of 53 or less, which is a 17-point descent. Hill climbing never accepts a downhill step, and it only looks one step ahead, so it cannot cross the valley. Once Kaito entered the left hill's basin, the climb was already doomed.

Echo Split (random restart) fixes this by changing the start, not the rule:

start	final	value	steps	which hill
(0,0)	(2,1)	70	3	left hill (false summit)
(4,0)	(2,1)	70	3	left hill, via (3,0)→(2,0)
(7,0)	**(6,6)**	**95**	**7**	**right ridge, straight along row 6**
(0,4)	(6,6)	95	8	right ridge, down column 6

RR_BEST_VALUE = 95, first reached from (7,0), with RR_TOTAL_STEPS = 21. A restart succeeds exactly when its start already lies in the summit's basin.

2. Gennai: why did only some fate scrolls reach the summit? *(simulated annealing)*

A downhill move of size Δ < 0 is accepted with probability $e^{\Delta/T}$, and $T_n = T_0 \cdot 0.97^n$. Escaping the false peak means chaining several −5 to −7 steps downhill to reach the valley floor (53, 17 below the peak). That is only likely while T is large compared with those steps.

iter	T (T_0=20)	p(Δ=−7)	T (T_0=40)	p(Δ=−7)
0	20.00	0.705	40.00	0.839
25	9.34	0.473	18.68	0.687
50	4.36	0.201	8.72	0.448
75	2.04	0.032	4.07	0.179
100	0.95	0.001	1.90	0.025
125	0.44	0.000	0.89	0.000

The run is decided early. With T_0 = 20, the chance of taking a −7 step falls below 1% at iteration 85. After that the walk is effectively hill climbing: uphill and level moves are always taken, so it climbs to the top of whichever hill it is on. Success therefore depends on *which basin the walker is in when it freezes*. The seed only decides the random path through those first ~85 iterations.

With T_0 = 20 that happens for **8 of the 20 seeds** (SA_SUCCESS_1_TO_20 = 8). The other 12 freeze on the left hill and end at (2,1) = 70. In every T_0 = 20 run the best value equals the final value, so no run touched the summit and then lost it.

Starting heat 40

Doubling T_0 shifts the whole schedule later by ln 2 / ln(1/0.97) ≈ 23 iterations. The 1% freeze point moves from iteration 85 to 108. Each run therefore makes more risky moves: 20–32 downhill moves accepted, against 12–20 at T_0 = 20. **Successes rise from 8/20 to 12/20.**

- Seeds 4, 8, 10, 13 and 19 now succeed. Only seed 6 fails where it used to succeed.
- Seed 6 (best 74) and seed 15 (best 81) crossed onto the right ridge but wandered back while the walk was still hot, so they ended at 70. Extra heat buys exploration but also lets a walker leave good ground.
- Seeds reach 95 later (first visit at iteration 20–108, against 20–73) and keep leaving it for longer (last exit up to iteration 106), because the walk stays restless for longer.

Heat and cooling together set how long the "hot" phase lasts compared with the 200-iteration budget. Extra sweep, successes out of 20:

cooling	T_0=5	10	20	30	40	60	80
0.95	0	1	3	6	8	8	11
0.97	**0**	**3**	**8**	**9**	**12**	**12**	**12**
0.98	1	5	13	12	17	14	16
0.99	3	11	10	8	7	6	2

If the start is too cold (T_0 = 5) or the cooling too fast (0.95), the walk freezes before it can cross the valley. With slow cooling (0.99), more heat makes results *worse* (2/20 at T_0 = 80): the walk is still hot at iteration 200 and ends wherever its random walk happens to stop. The best results need enough heat to cross the valley *and* enough cold iterations afterwards to climb the summit.

3. Rin: why did my seeds stop at 234 when 241 was possible? *(genetic algorithm)*

bit	tool	load	vol	use	GA	OPT
0	Kunai set	18	9	36	1	1
2	**Smoke bombs**	**12**	**5**	**10**	**0**	**1**
3	Paper bombs	9	3	32	1	1
4	Energy pills	5	7	31	1	1
5	**Flash tags**	**5**	**6**	**8**	**1**	**0**
6	Messenger scroll	11	3	33	1	1
9	Wind shuriken	11	2	35	1	1
10	**Twin blades**	**14**	**6**	**21**	**1**	**0**
12	**Medical kit**	**8**	**8**	**26**	**0**	**1**
14	Binding tags	10	7	38	1	1

Bits 1, 7, 8, 11, 13 are 0 in both pouches.

GA (gen 50): `100111100110001`, load 83/86, volume 43/44, **234**.
Optimum (brute force over all 32,768 pouches, unique): `101110100100101`, load 84/86, volume 44/44, **241**.

Seed by seed, the pouches differ in exactly four seeds. The GA keeps Flash tags and Twin blades (load 19, volume 12, use 29). The optimum packs Smoke bombs and Medical kit instead (load 20, volume 13, use 36). The swap gains 7 usefulness for 1 more load and 1 more volume, which fills both limits exactly.

Why the seeds missed it:

- **234 is a local optimum in bit space.** Each of the 15 one-bit changes either breaks a limit (fitness 0) or scores 196–226. Brute force shows that 241 is the *only* feasible pouch better than 234, and it is exactly four flips away. Mutation flips each bit with p = 1/15, so a child gets precisely those four flips with probability $(1/15)^4(14/15)^{11} \approx 9\times10^{-6}$. Over the ~630 children bred after generation 15, that is under 1%.
- **The fitness cliff blocks gradual paths.** Adding Smoke bombs or the Medical kit first overflows a limit and scores 0. So every intermediate step toward 241 is worthless and selection discards it.
- **Premature convergence removed the missing building blocks.** The GA reached 234 at generation 15 (GA_FIRST_GEN) and never improved. Average pairwise Hamming distance fell from 7.38 (gen 0) to 2.26 (gen 50). In the final population, bits 5 and 10 are set in 18 and 19 of the 20 chromosomes, but bits 2 and 12 (the ones 241 needs) appear in only 1 each. Crossover cannot assemble genes the population no longer carries. A population of 20 with two elites and size-3 tournaments applies strong selection pressure, so a single good pouch quickly takes over.

One fix: a repair operator (plus a little more mutation). After mutation, if a child breaks a limit, unpack its least valuable tool (lowest usefulness per unit of scaled load + volume) until it fits. This removes the cliff: an "add Medical kit" mutation becomes "add Medical kit, drop Flash tags", which is a feasible step toward 241. Extra runs over seeds 1–30, measuring the best feasible pouch in the last generation:

variant	found 241	mean best	worst
as specified (Owl Cipher settings)	14/30	236.9	230
100 generations	18/30	238.1	232
mutation 2/15 or 3/15 alone	11/30, 10/30	236.1, 234.3	232, 224
penalty fitness (use − w·overflow, w = 3 or 10)	3/30, 4/30	226.1, 230.1	214, 217
greedy repair	22/30	238.9	232
greedy repair + mutation 2/15	**29/30**	**240.7**	**232**

A penalty alone did worse: the population settled just over the limits, where usefulness is high but the pouch is illegal. Repair keeps every member legal, and the higher mutation rate helps restore lost diversity.

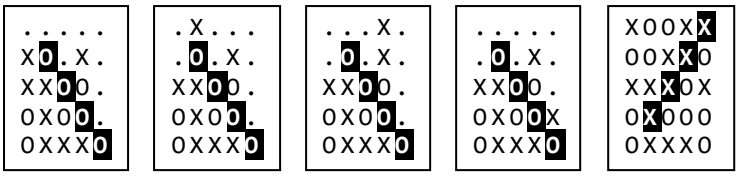
4. Vesper: for every move that would have lost, show how I would have beaten you *(minimax)*

Full minimax (column order 0–4) gives the root value **+1** with best move **2**. Each root column has an exact value:

X plays	0	1	2	3	4
value	−10	−10	**+1**	−10	−10

All four losing moves lose the same way, at once. O already holds (3,1), (1,3) and (0,4) on the falling diagonal. The missing cell is (2,2), which is exactly where the next stone in column 2 lands. If X plays anywhere except column 2, **Vesper drops into column 2** and completes four with 9 cells still empty, so the utility is −(1+9) = −10. X has no immediate win of its own, so nothing X does

elsewhere comes first. The principal variation for each loss is just two plies: X0/X1/X3/X4, then O2. Final positions (row 4 at the top; **inverted cells** mark O's line):



X0, O2 → −10 X1, O2 → −10 X3, O2 → −10 X4, O2 → −10 X2 … X4 → +1

The winning move 2 blocks (2,2) with X. After that, O has no immediate win. Best play by both sides then runs `X2 O0 X0 O1 X2 O2 X3 O4 X4 O4 X4`: X fills the last cell and completes the rising diagonal (1,1)–(2,2)–(3,3)–(4,4) (inverted), using the stone from move 2. Since no cells are left, the utility is 1 + 0 = +1. Vesper can hold out to the final stone, but cannot avoid losing.

5. Jun: compare the ki of every method *(minimax vs alpha-beta)*

run	value	move	nodes	leaves	saved
MM: full minimax, order 01234	1	2	175,789	64,066	n/a
AB1: alpha-beta, full, 01234	1	2	9,064	2,791	94.8% vs MM
AB2: alpha-beta, full, 21304	1	2	8,965	2,976	94.9% vs MM
DL: minimax, depth 4, 01234	−2	2	421	47	n/a
AB3: alpha-beta, depth 4, 01234	−2	2	198	8	53.0% vs DL

Alpha-beta returns the same value and move as minimax every time, as pruning must. Full alpha-beta needs about 1/19 of the nodes of full minimax. The depth-4 searches are far cheaper still, but they are only estimates: DL/AB3 value the position at −2 (O's open lines look stronger), when the true value is +1. They still pick column 2, because blocking O's diagonal ranks highest even with a 4-move horizon.

Why did middle-first barely help? (AB2 saves only 99 nodes, 1.1%)

root col	minimax	AB 01234	AB 21304
0	20,923	1,592	2
1	12,564	330	2
2	**81,721**	**6,781**	**8,956**
3	10,758	103	2
4	49,822	257	2
nodes in each root move's subtree (the root adds 1)			

Middle-first does exactly what it promises, *where the best move really is in the middle*. Vesper's refutation of every losing X move is O2, so with column 2 first each losing move is refuted in **2 nodes**. That saves 2,274 nodes against the 01234 order (1,592 + 330 + 103 + 257 − 8).

But most of the work (75% of AB1's nodes) is inside the one move that matters, X2, and there middle-first is *worse*: **8,956 nodes against 6,781** (+2,175). After X2 the "middle" is nearly gone. Columns 1 and 3 have one empty cell each and column 2 has two, and the main line opens `O0 X0 O1`. Putting the middle columns first there does not bring strong replies forward, and the measured cost goes up because weaker moves are tried before the cut-offs arrive. The gain at the root and the loss inside X2 almost cancel: 2,274 − 2,175 = 99.

Across all 120 column orders, full alpha-beta ranges from 2,284 to 29,284 nodes (median 8,188). Middle-first ranks 65th of 120 and 01234 ranks 66th. The best fixed order (13240) needs 2,284. Move ordering matters a great deal, but a single "middle first" rule ignores where this position's strong moves actually are.

Master Hyo's tree: left-to-right trace, and why right-to-left read far fewer leaves

node	receives (α,β)	value	what happened
A1 (MAX)	(−∞,∞)	−15	A1a=−27; A1b: L4=−28 ≤ α → cut L5,L6; A1c=−20; A1d=−15
A2 (MAX)	(−∞,−15)	0	A2a=−29, A2b=−22, A2c=0 ≥ β (last child)
A3 (MAX)	(−∞,−15)	−15	A3a=−15 ≥ β → cut A3b, A3c, A3d (7 leaves)
A (MIN)	(−∞,∞)	−15	root α becomes −15
B1 (MAX)	(−15,∞)	26	B1a=−10, B1b=−3, B1c=−18 (L38 last), B1d=26
B2 (MAX)	**(−15,26)**	**9**	**B2a=9; B2b: L44=−3 ≤ 9 → cut L45; B2c: L46 → cut L47,L48**
B (MIN)	(−15,∞)	9	root α becomes 9, best move B
C1 (MAX)	(9,∞)	5	C1a: L49=5 ≤ 9 cut; C1b: L52 cut; C1c: L54 cut
C (MIN)	(9,∞)	5	5 ≤ α=9 → cut C2 (5 leaves)
R (MAX)	(−∞,∞)	**9**	TREE_MOVE = B, 39 leaves read

Exact values of the root's children: A = −15, **B = 9**, C = −4. Left to right reads **39 leaves** (A 20/29, B 16/19, C 3/12). Right to left reads only **23** (C 10/12, B 8/19, A 5/29), for the same value 9. Pruning needs good bounds *early*, and the right-hand side of this tree supplies them:

- **A better opening bound at the root.** Left to right starts with A, the worst child (−15), so B is searched with α = −15, a weak bound. Right to left starts with C = −4, so α = −4 is already tighter before B.
- **Inside B, the decisive child comes first.** B's value is set by B2 (9). Right to left reaches B2 before B1, so β = 9 is known when B1 starts. B1d = 26 ≥ 9 cuts B1c, B1b and B1a at once (only 2 of B1's 11 leaves read). Left to right read all 11 of B1's leaves before B2 gave anything useful.
- **A is dismissed cheaply.** When right to left reaches A, α = 9. Each grandchild of A3 is cut after its first leaf (L29 = −28, L27 = 4, L23 = −29 after L24, L22 = 7, all ≤ 9), so A3 = 7 ≤ 9 and A2 and A1 are never touched: 19 leaves skipped. Left to right had to search A first with no bound at all.

This is the same lesson as Stone Drop: alpha-beta's saving comes from meeting each node's strongest reply first. In this tree, the strong values (C's −4, B2's 9) sit on the right, so right-to-left is the near-ideal order. In Stone Drop, a fixed rule like "middle first" only helps where the strong moves really are in the middle.